

```

import { useState, useRef, useEffect } from "react";

const MODEL = "claude-sonnet-4-20250514";

/* ——— utils ————— */
function daysFromNow(n) {
  const d = new Date();
  d.setDate(d.getDate() + n);
  return d.toISOString().split("T")[0];
}
function todayStr() { return new Date().toISOString().split("T")[0]; }
function daysLeft(s) { return Math.ceil((new Date(s) - new Date()) / 86400000); }
function expiryColor(s) {
  const d = daysLeft(s);
  return d <= 3 ? "#C53030" : d <= 7 ? "#B7791F" : "#276749";
}
function expiryLabel(s) {
  const d = daysLeft(s);
  if (d <= 0) return "Abgelaufen";
  if (d === 1) return "Morgen";
  return `${d} Tage`;
}
function greet() {
  const h = new Date().getHours();
  if (h < 12) return "Guten Morgen";
  if (h < 18) return "Guten Tag";
  return "Guten Abend";
}

/* ——— seed data ————— */
const SEED_ITEMS = [
  { id:1, name:"Vollmilch", cat:"Milchprodukte", qty:"1", unit:"L", ex
  { id:2, name:"Emmentaler", cat:"Milchprodukte", qty:"200", unit:"g", ex
  { id:3, name:"Bio-Eier", cat:"Milchprodukte", qty:"6", unit:"Stk", ex
  { id:4, name:"Hühnerbrust", cat:"Fleisch", qty:"500", unit:"g", ex
  { id:5, name:"Spaghetti", cat:"Pasta & Reis", qty:"400", unit:"g", ex
  { id:6, name:"Tomatensauce", cat:"Konserven", qty:"2", unit:"Dosen",ex
  { id:7, name:"Zwiebeln", cat:"Gemüse", qty:"3", unit:"Stk", ex
  { id:8, name:"Knoblauch", cat:"Gemüse", qty:"1", unit:"Kopf", ex
  { id:9, name:"Olivenöl", cat:"Öle", qty:"500", unit:"ml", ex
  { id:10, name:"Joghurt Natur", cat:"Milchprodukte", qty:"2", unit:"Stk", ex
];

const SEED_RECIPES = [

```

```

{
  id: 1,
  name: "Spaghetti Aglio e Olio",
  time: 20,
  diff: "Einfach",
  diet: "vegetarisch",
  ing: ["400g Spaghetti", "4 Knoblauchzehen", "6 EL Olivenöl", "Peperoncino", "
  steps: [
    "Spaghetti in reichlich Salzwasser bissfest kochen.",
    "Knoblauch in Scheiben schneiden, in Olivenöl bei mittlerer Hitze goldbraun
    "Peperoncino hinzufügen, kurz mitrösten.",
    "Pasta abgiessen, etwas Kochwasser behalten. Alles vermengen und abschmecke
  ],
  gen: false,
},
];

```

```

/* — design tokens ————— */

```

```

const C = {
  bg:      "#F9F7F3",
  card:    "#FFFFFF",
  card2:   "#F2EFE9",
  border:  "#E9E4DC",
  text:    "#1A1714",
  sub:     "#7C746C",
  green:   "#2D7D46",
  greenL:  "#EBF5EE",
  greenM:  "#C6E6CE",
  orange:  "#C05621",
  orangeL: "#FEF0E7",
  red:     "#C53030",
  redL:    "#FEF2F2",
  amber:   "#B7791F",
  purple:  "#6B46C1",
};

```

```

/* — icons ————— */

```

```

const ICONS = {
  home:      "M3 9.5L12 3l9 6.5V20a1 1 0 01-1 1H5a1 1 0 01-1-1V9.5z M9 21V12h6v9",
  pantry:    "M21 16V8a2 2 0 00-1-1.73l-7-4a2 2 0 00-2 01-7 4A2 2 0 003 8v8a2 2 0
  chef:      "M12 2C8.13 2 5 5.13 5 9c0 2.38 1.19 4.47 3 5.74V17c0 .55.45 1 1 1h6c
  settings:  "M12 15a3 3 0 100-6 3 3 0 000 6z M19.4 15a1.65 1.65 0 00.33 1.821.06.
  plus:      "M12 5v14M5 12h14",
  trash:     "M3 6h18M19 6v14a2 2 0 01-2 2H7a2 2 0 01-2-2V6M8 6V4a2 2 0 012-2h4a2
  edit:      "M11 4H4a2 2 0 00-2 2v14a2 2 0 002 2h14a2 2 0 002-2v-7M18.5 2.5a2.121
  clock:     "M12 22c5.52 0 10-4.48 10-10S17.52 2 12 2 2 6.48 2 12s4.48 10 10 10z
  fire:      "M12 23c-4.97 0-9-3.58-9-8 0-3.31 1.99-5.86 4-7.5.29 1.77 1.5 3.37 3

```

```

x:          "M18 6L6 18M6 6l12 12",
sparkle:    "M12 212.4 7.4H221-6.2 4.5 2.4 7.4L12 171-6.2 4.3 2.4-7.4L2 9.4h7.6L1
leaf:       "M2 22c3-3 6-8 6-13 3 0 7 2 9 5-1-3 0-6 2-8 1 6-1 11-5 14-1 1-2 2-4 2
camera:     "M23 19a2 2 0 01-2 2H3a2 2 0 01-2-2V8a2 2 0 012-2h4l2-3h6l2 3h4a2 2 0
warning:    "M10.29 3.86L1.82 18a2 2 0 001.71 3h16.94a2 2 0 001.71-3L13.71 3.86a2
search:     "M21 211-4.35-4.35M17 11A6 6 0 115 11a6 6 0 0112 0z",
check:      "M20 6L9 171-5-5",
receipt:    "M9 5H7a2 2 0 00-2 2v12a2 2 0 002 2h10a2 2 0 002-2V7a2 2 0 00-2-2h-2M
star:       "M12 213.09 6.26L22 9.271-5 4.87 1.18 6.88L12 17.771-6.18 3.25L7 14.1
info:       "M12 22c5.52 0 10-4.48 10-10S17.52 2 12 2 2 6.48 2 12s4.48 10 10 10z
};

```

```

function Ic({ n, s = 22, col = "currentColor", sw = 1.8 }) {
  return (
    <svg width={s} height={s} viewBox="0 0 24 24" fill="none"
      stroke={col} strokeWidth={sw} strokeLinecap="round" strokeLinejoin="round"
      style={{ flexShrink: 0 }}>
      <path d={ICONS[n] || ""} />
    </svg>
  );
}

```

/* — small reusable pieces ————— */

```

function Chip({ icon, text, col }) {
  return (
    <div style={{ display:"inline-flex", alignItems:"center", gap:5,
      background:`${col}18`, border:`1px solid ${col}30`,
      borderRadius:20, padding:"4px 10px" }}>
      {icon} && <Ic n={icon} s={11} col={col} />
      <span style={{ fontSize:11, fontWeight:700, color:col }}>{text}</span>
    </div>
  );
}

```

```

function SectionLabel({ icon, col, label }) {
  return (
    <div style={{ display:"flex", alignItems:"center", gap:7, marginBottom:12 }}>
      <Ic n={icon} s={14} col={col} />
      <span style={{ fontSize:11, fontWeight:700, color:C.sub,
        textTransform:"uppercase", letterSpacing:"0.09em" }}>{label}</span>
    </div>
  );
}

```

```

function FieldLabel({ children }) {
  return <div style={{ fontSize:12, fontWeight:700, color:C.sub, marginBottom:6 }}
}

```

```

function PrimaryBtn({ onClick, text, col, disabled, loading }) {
  return (
    <button onClick={onClick} disabled={disabled || loading} className="tap"
      style={{ width:"100%", background: (disabled||loading) ? C.card2 : col,
        border:"none", borderRadius:14, padding:"15px", fontSize:14,
        fontWeight:800, color:(disabled||loading) ? C.sub : "#fff",
        cursor:(disabled||loading) ? "not-allowed" : "pointer",
        display:"flex", alignItems:"center", justifyContent:"center", gap:8,
        boxShadow:(disabled||loading) ? "none" : `0 3px 12px ${col}35`,
        transition:"all .2s" }}>
      {loading ? <><Spinner white /><span>Lädt...</span></> : text}
    </button>
  );
}

function Spinner({ white }) {
  const bc = white ? "rgba(255,255,255,0.3)" : C.border;
  const tc = white ? "#fff" : C.green;
  return (
    <div style={{ width:18, height:18,
      border:`2.5px solid ${bc}`, borderTopColor:tc,
      borderRadius:"50%", animation:"spin 0.9s linear infinite" }} />
  );
}

function ErrBox({ msg }) {
  return (
    <div style={{ background:C.redL, border:`1px solid ${C.red}30`,
      borderRadius:12, padding:"11px 14px", fontSize:13, color:C.red,
      marginBottom:14, lineHeight:1.5 }}>{msg}</div>
  );
}

/* shared input style — function, not broken default arg */
function inputStyle(small) {
  return {
    width:"100%", background:C.card2, border:`1px solid ${C.border}`,
    borderRadius:12, padding: small ? "10px 10px" : "12px 13px",
    fontSize: small ? 12 : 14, outline:"none", color:C.text,
    fontFamily:"inherit",
  };
}

/* — sheet (bottom modal) ————— */
function Sheet({ onClose, title, children }) {
  return (

```

```

<div
  style={{ position:"fixed", inset:0, zIndex:100, display:"flex",
    alignItems:"flex-end", background:"rgba(0,0,0,0.32)", backdropFilter:"blur(2px)",
    onClick={e => { if (e.target === e.currentTarget) onClose(); }}>
    <div className="slide-up"
      style={{ width:"100%", maxWidth:430, margin:"0 auto", background:C.card,
        borderRadius:"24px 24px 0 0",
        padding:`20px 20px calc(24px + env(safe-area-inset-bottom,0px))`,
        maxHeight:"92dvh", overflowY:"auto",
        boxShadow:"0 -8px 32px rgba(0,0,0,0.10)" }}>
      <div style={{ display:"flex", justifyContent:"space-between",
        alignItems:"center", marginBottom:20 }}>
        <div style={{ fontSize:17, fontWeight:800 }}>{title}</div>
        <button onClick={onClose} className="tap"
          style={{ background:C.card2, border:`1px solid ${C.border}`,
            borderRadius:10, padding:"7px", cursor:"pointer", display:"flex" }}
          <Ic n="x" s={16} col={C.sub} />
        </button>
      </div>
      {children}
    </div>
  </div>
);
}

```

```

/* ————— bottom nav ————— */

```

```

function BottomNav({ tab, setTab }) {
  const tabs = [
    { id:"home", icon:"home", label:"Start" },
    { id:"pantry", icon:"pantry", label:"Vorrat" },
    { id:"recipes", icon:"chef", label:"Rezepte" },
    { id:"settings", icon:"settings", label:"Einstellungen" },
  ];
  return (
    <nav style={{ position:"fixed", bottom:0, left:"50%",
      transform:"translateX(-50%)", width:"100%", maxWidth:430,
      background:C.card, borderTop:`1px solid ${C.border}`, display:"flex",
      paddingBottom:"env(safe-area-inset-bottom,0px)", zIndex:50,
      boxShadow:"0 -1px 14px rgba(0,0,0,0.06)" }}>
      {tabs.map(t => (
        <button key={t.id} onClick={() => setTab(t.id)} className="tap"
          style={{ flex:1, padding:"12px 4px 10px", border:"none",
            background:"transparent", cursor:"pointer", display:"flex",
            flexDirection:"column", alignItems:"center", gap:4 }}>
          <Ic n={t.icon} s={22} col={tab === t.id ? C.green : C.sub} />
          <span style={{ fontSize:10, fontWeight:700,
            color:tab === t.id ? C.green : C.sub }}>{t.label}</span>
        </button>
      ))}
    </nav>
  );
}

```

```

        </button>
    )))
</nav>
);
}

/* =====
HOME TAB
===== */
function HomeTab({ items, recipes, soon, setTab, setModal }) {
    const daily = recipes[0] || null;
    return (
        <div style={{ padding:"20px 20px 0" }} className="fu">

            {/* daily recipe */}
            <div style={{ marginBottom:20 }}>
                <SectionLabel icon="star" col={C.amber} label="Tagesrezept" />
                {daily ? (
                    <div onClick={() => setModal({ type:"view", data:daily })} className="t
                        style={{ background:`linear-gradient(135deg,${C.green},#1a5c32)`,
                            borderRadius:20, padding:"20px", cursor:"pointer",
                            boxShadow:`0 6px 24px ${C.green}40`, position:"relative", overflow:
                    <div style={{ position:"absolute", right:-20, top:-20, width:100, hei
                        borderRadius:"50%", background:"rgba(255,255,255,0.07)" }} />
                    <div style={{ position:"absolute", right:10, bottom:-30, width:140, h
                        borderRadius:"50%", background:"rgba(255,255,255,0.04)" }} />
                    <div style={{ fontSize:11, fontWeight:700, color:"rgba(255,255,255,0.
                        textTransform:"uppercase", letterSpacing:"0.1em", marginBottom:6 }}
                    <div style={{ fontSize:20, fontWeight:800, color:"#fff",
                        marginBottom:10, lineHeight:1.2 }}>{daily.name}</div>
                    <div style={{ display:"flex", gap:8 }}>
                        <div style={{ display:"flex", alignItems:"center", gap:5,
                            background:"rgba(255,255,255,0.15)", borderRadius:20, padding:"5p
                            <Ic n="clock" s={13} col="rgba(255,255,255,0.9)" />
                            <span style={{ fontSize:12, color:"rgba(255,255,255,0.9)", fontWe
                        </div>
                        {daily.diet && (
                            <div style={{ display:"flex", alignItems:"center", gap:5,
                                background:"rgba(255,255,255,0.15)", borderRadius:20, padding:"
                                <Ic n="leaf" s={13} col="rgba(255,255,255,0.9)" />
                                <span style={{ fontSize:12, color:"rgba(255,255,255,0.9)", font
                            </div>
                        ))
                    </div>
                    <div style={{ marginTop:10, fontSize:12, color:"rgba(255,255,255,0.6)
                        {daily.ing.slice(0,3).join(" · ")}
                    </div>
                )}
            </div>
        </div>
    )}

```

```

    </div>
  ) : (
    <div onClick={() => setModal({ type:"ai" })} className="tap"
      style={{ background:C.greenL, border:`1.5px dashed ${C.greenM}`,
        borderRadius:20, padding:"24px", textAlign:"center", cursor:"pointe
    <Icon n="sparkle" s={28} col={C.green} />
    <div style={{ fontSize:14, fontWeight:700, color:C.green, marginTop:8
    <div style={{ fontSize:12, color:C.sub, marginTop:4 }}>Noch kein Tage
    </div>
  )}
</div>

{/* expiring soon */}
{soon.length > 0 && (
  <div style={{ marginBottom:20 }}>
    <SectionLabel icon="fire" col={C.red} label="Läuft bald ab" />
    {soon.slice(0,3).map((item,i) => (
      <div key={item.id} className="fu"
        style={{ animationDelay:`${i*0.06}s`, background:C.card,
          border:`1px solid ${C.border}`, borderRadius:14,
          padding:"12px 14px", marginBottom:8, display:"flex",
          justifyContent:"space-between", alignItems:"center",
          boxShadow:"0 1px 3px rgba(0,0,0,0.06)" }}>
        <div>
          <div style={{ fontWeight:700, fontSize:14 }}>{item.name}</div>
          <div style={{ fontSize:11, color:expiryColor(item.expiry),
            fontWeight:700, marginTop:2 }}>{expiryLabel(item.expiry)}</div>
        </div>
        <div style={{ background:`${expiryColor(item.expiry)}18`,
          border:`1px solid ${expiryColor(item.expiry)}30`,
          borderRadius:20, padding:"4px 10px" }}>
          <span style={{ fontSize:12, color:expiryColor(item.expiry), fontW
            {item.qty} {item.unit}
          </span>
        </div>
      </div>
    ))}
    {soon.length > 3 && (
      <button onClick={() => setTab("pantry")} className="tap"
        style={{ width:"100%", background:C.card2, border:`1px solid ${C.bo
          borderRadius:12, padding:"10px", fontSize:13, fontWeight:600,
          color:C.sub, cursor:"pointer" }}>
        +{soon.length - 3} weitere ansehen
      </button>
    )}
  </div>
)}

```

```

    {/* quick tiles */}
    <div style={{ marginBottom:20 }}>
      <SectionLabel icon="pantry" col={C.sub} label="Übersicht" />
      <div style={{ display:"grid", gridTemplateColumns:"1fr 1fr", gap:10 }}>
        <div onClick={() => setTab("recipes")} className="tap"
          style={{ background:C.card, border:`1px solid ${C.border}`,
            borderRadius:16, padding:"16px", cursor:"pointer",
            boxShadow:"0 1px 3px rgba(0,0,0,0.06)" }}>
          <Ic n="chef" s={22} col={C.green} />
          <div style={{ fontSize:28, fontWeight:900, letterSpacing:"-1px",
            marginTop:8, color:C.text }}>{recipes.length}</div>
          <div style={{ fontSize:12, color:C.sub, fontWeight:600, marginTop:2 }}
        </div>
        <div onClick={() => setModal({ type:"receipt" })} className="tap"
          style={{ background:`linear-gradient(135deg,${C.orange},#9C4221)`,
            border:"none", borderRadius:16, padding:"16px", cursor:"pointer",
            boxShadow:`0 4px 16px ${C.orange}30`, position:"relative", overflow
          <div style={{ position:"absolute", right:-10, bottom:-10,
            width:60, height:60, borderRadius:"50%", background:"rgba(255,255,2
          <Ic n="receipt" s={22} col="#fff" />
          <div style={{ fontSize:14, fontWeight:800, color:"#fff", marginTop:8
          <div style={{ fontSize:11, color:"rgba(255,255,255,0.75)", marginTop:
            </div>
          </div>
        </div>
      </div>
    </div>
  );
}

/* =====
PANTRY TAB
===== */

function PantryTab({ items, removeItem, setModal }) {
  const [q, setQ] = useState("");
  const [cat, setCat] = useState("all");
  const cats = ["all", ...new Set(items.map(i => i.cat))];
  const list = items
    .filter(i => (cat === "all" || i.cat === cat) && i.name.toLowerCase().include
    .sort((a, b) => daysLeft(a.expiry) - daysLeft(b.expiry));

  return (
    <div className="fu">
      <div style={{ padding:"14px 20px 0" }}>
        <div style={{ display:"flex", gap:8, marginBottom:11 }}>
          <div style={{ flex:1, position:"relative", display:"flex", alignItems:"
            <div style={{ position:"absolute", left:11, pointerEvents:"none", dis

```



```

        <Ic n="search" s={16} col={C.sub} />
    </div>
    <input value={q} onChange={e => setQ(e.target.value)}
      placeholder="Suchen..."
      style={{ width:"100%", background:C.card, border:`1px solid ${C.bor
        borderRadius:12, padding:"11px 12px 11px 36px", fontSize:14,
        outline:"none", fontFamily:"inherit", color:C.text }} />
  </div>
  <button onClick={() => setModal({ type:"receipt" })} className="tap"
    style={{ background:C.green, border:"none", borderRadius:12,
      padding:"0 14px", cursor:"pointer", display:"flex",
      alignItems:"center", gap:6, boxShadow:`0 2px 8px ${C.green}30` }}>
    <Ic n="receipt" s={17} col="#fff" />
    <span style={{ fontSize:12, fontWeight:700, color:"#fff", whiteSpace:
  </button>
  <button onClick={() => setModal({ type:"addItem" })} className="tap"
    style={{ background:C.orange, border:"none", borderRadius:12,
      padding:"0 13px", cursor:"pointer", display:"flex",
      alignItems:"center", boxShadow:`0 2px 8px ${C.orange}30` }}>
    <Ic n="plus" s={20} col="#fff" />
  </button>
</div>

<div style={{ display:"flex", gap:7, overflowX:"auto", paddingBottom:12 }}
  {cats.map(ct => (
    <button key={ct} onClick={() => setCat(ct)} className="tap"
      style={{ flexShrink:0, background:cat===ct ? C.green : C.card,
        border:`1px solid ${cat===ct ? C.green : C.border}`,
        borderRadius:20, padding:"6px 14px", fontSize:12, fontWeight:700,
        color:cat===ct ? "#fff" : C.sub, cursor:"pointer", fontFamily:"in
        {ct === "all" ? "Alle" : ct}
      </button>
    ))}
  </div>
</div>

<div style={{ padding:"0 20px" }}>
  {list.length === 0 && (
    <div style={{ textAlign:"center", padding:"50px 0" }}>
      <Ic n="pantry" s={40} col={C.border} />
      <div style={{ color:C.sub, fontSize:14, marginTop:12 }}>Nichts gefund
    </div>
  )}
  {list.map((item, i) => (
    <div key={item.id} className="fu"
      style={{ animationDelay:`${i*0.04}s`, background:C.card,
        border:`1px solid ${C.border}`, borderRadius:14,

```

```

padding:"13px 14px", marginBottom:8, display:"flex",
justifyContent:"space-between", alignItems:"center",
boxShadow:"0 1px 3px rgba(0,0,0,0.06)" }}>
<div style={{ flex:1, minWidth:0 }}>
  <div style={{ fontWeight:700, fontSize:14,
    overflow:"hidden", textOverflow:"ellipsis", whiteSpace:"nowrap" }}
  <div style={{ fontSize:12, color:C.sub, marginTop:1 }}>
    {item.cat} · {item.qty} {item.unit}
  </div>
  <div style={{ marginTop:6, display:"flex", alignItems:"center", gap
    <div style={{ width:7, height:7, borderRadius:"50%",
      background:expiryColor(item.expiry), flexShrink:0 }} />
    <span style={{ fontSize:11, color:expiryColor(item.expiry), fontW
      {expiryLabel(item.expiry)}
    </span>
  </div>
</div>
<div style={{ display:"flex", gap:6, marginLeft:10 }}>
  <button onClick={() => setModal({ type:"addItem", edit:item })} cla
    style={{ background:C.card2, border:`1px solid ${C.border}`,
      borderRadius:9, padding:"8px", cursor:"pointer", display:"flex"
    <Ic n="edit" s={15} col={C.sub} />
  </button>
  <button onClick={() => removeItem(item.id)} className="tap"
    style={{ background:C.redL, border:`1px solid ${C.red}25`,
      borderRadius:9, padding:"8px", cursor:"pointer", display:"flex"
    <Ic n="trash" s={15} col={C.red} />
  </button>
</div>
</div>
  )}}
</div>
</div>
);
}

```

```

/* =====
  RECIPES TAB
  ===== */

```

```

function RecipesTab({ recipes, items, setModal }) {
  const [diet, setDiet] = useState("all");
  const diets = ["all","vegetarisch","vegan","fleisch"];
  const list = recipes.filter(r => diet === "all" || r.diet === diet);
  const hasEnough = items.length >= 3;

  return (
    <div className="fu">

```

```

<div style={{ padding:"14px 20px 0" }}>

  {hasEnough ? (
    <button onClick={() => setModal({ type:"ai" })} className="tap"
      style={{ width:"100%", background:`linear-gradient(135deg,${C.green},
        border:"none", borderRadius:16, padding:"17px", cursor:"pointer",
        display:"flex", alignItems:"center", justifyContent:"center", gap:1
        marginBottom:10, boxShadow:`0 4px 18px ${C.green}38`, fontFamily:"i
      <Ic n="sparkle" s={20} col="#fff" />
      <span style={{ fontSize:15, fontWeight:800, color:"#fff" }}>KI-Rezept
    </button>
  ) : (
    <div style={{ background:C.orangeL, border:`1.5px solid ${C.orange}30`,
      borderRadius:16, padding:"16px", marginBottom:10, display:"flex",
      gap:12, alignItems:"flex-start" }}>
      <div style={{ marginTop:2 }}><Ic n="info" s={18} col={C.orange} /></d
    <div>
      <div style={{ fontWeight:700, fontSize:14, color:C.orange, marginBo
      <div style={{ fontSize:13, color:C.sub, lineHeight:1.6 }}>
        Du hast erst <strong>{items.length}</strong> {items.length === 1
        Füge mindestens 3 Lebensmittel hinzu, damit die KI ein sinnvolles
      </div>
    </div>
  </div>
  )}

  <button onClick={() => setModal({ type:"addRecipe" })} className="tap"
    style={{ width:"100%", background:C.card, border:`1px solid ${C.border}
    borderRadius:13, padding:"12px", cursor:"pointer", display:"flex",
    alignItems:"center", justifyContent:"center", gap:7, marginBottom:12,
    boxShadow:"0 1px 3px rgba(0,0,0,0.06)", fontFamily:"inherit" }}>
    <Ic n="plus" s={17} col={C.orange} />
    <span style={{ fontSize:13, fontWeight:700 }}>Eigenes Rezept hinzufügen
  </button>

  <div style={{ display:"flex", gap:7, overflowX:"auto", paddingBottom:12 }
    {diets.map(d => (
      <button key={d} onClick={() => setDiet(d)} className="tap"
        style={{ flexShrink:0, background:diet===d ? C.orange : C.card,
        border:`1px solid ${diet===d ? C.orange : C.border}`,
        borderRadius:20, padding:"6px 14px", fontSize:12, fontWeight:700,
        color:diet===d ? "#fff" : C.sub, cursor:"pointer", fontFamily:"in
        {d === "all" ? "Alle" : d.charAt(0).toUpperCase() + d.slice(1)}
      </button>
    )}}
  </div>
</div>

```

```

<div style={{ padding:"0 20px" }}>
  {list.length === 0 && (
    <div style={{ textAlign:"center", padding:"50px 0" }}>
      <Icon n="chef" s={40} col={C.border} />
      <div style={{ color:C.sub, fontSize:14, marginTop:12 }}>Noch keine Re
    </div>
  )}
  {list.map((r, i) => (
    <div key={r.id} onClick={() => setModal({ type:"view", data:r })} class
      style={{ animationDelay:`${i*0.05}s`, background:C.card,
        border:`1px solid ${C.border}`, borderRadius:16, padding:"16px",
        marginBottom:10, cursor:"pointer", boxShadow:"0 1px 3px rgba(0,0,0,
    <div style={{ fontWeight:800, fontSize:15, marginBottom:8 }}>{r.name}
    <div style={{ display:"flex", gap:7, flexWrap:"wrap" }}>
      <Chip icon="clock" text={` ${r.time} Min`} col={C.orange} />
      <Chip text={r.diff} col={C.sub} />
      {r.diet && <Chip icon="leaf" text={r.diet} col={C.green} />}
      {r.gen && <Chip icon="sparkle" text="KI" col={C.purple} />}
    </div>
    <div style={{ marginTop:10, fontSize:12, color:C.sub, lineHeight:1.5
      {r.ing.slice(0,4).join(" · ")}
      {r.ing.length > 4 ? ` +${r.ing.length - 4}` : ""}
    </div>
  </div>
  )})
</div>
</div>
);
}

```

```

/* =====
SETTINGS TAB
===== */

function SettingsTab() {
  return (
    <div style={{ padding:"20px" }} className="fu">
      <SectionLabel icon="settings" col={C.orange} label="Einstellungen" />
      <div style={{ background:C.card, border:`1px solid ${C.border}`, borderRadi
        padding:"18px", boxShadow:"0 1px 3px rgba(0,0,0,0.06)", marginBottom:12 }
      <div style={{ fontWeight:700, fontSize:14, marginBottom:8 }}>Über Freshly
      <div style={{ fontSize:13, color:C.sub, lineHeight:1.8 }}>
        Version 2.0<br />
        KI-gestützte Lebensmittelverwaltung<br />
        Kassenzettel-Scan mit Kamera und KI<br />
        Optimiert für Schweizer Supermärkte
      </div>
    </div>
  )
}

```

```

    </div>
    <InfoBox icon="camera" title="Kassenzettel scannen" col={C.green}>
      Fotografiere deinen Kassenzettel. Die KI liest alle Lebensmittel automati
      und filtert Nicht-Lebensmittel (Zahnpasta, Reinigungsmittel usw.) heraus.
    </InfoBox>
    <InfoBox icon="receipt" title="Kamera-Zugriff" col={C.orange}>
      Der Browser fragt einmalig nach Erlaubnis für die Kamera.
      Auf dem iPhone am besten in Safari öffnen.
    </InfoBox>
  </div>
);
}

```

```

function InfoBox({ icon, title, col, children }) {
  return (
    <div style={{ background:`${col}0D`, border:`1px solid ${col}28`,
      borderRadius:16, padding:"16px", marginBottom:10,
      display:"flex", gap:12, alignItems:"flex-start" }}>
      <div style={{ marginTop:1, flexShrink:0 }}><Ic n={icon} s={18} col={col} />
      <div>
        <div style={{ fontWeight:700, fontSize:13, color:col, marginBottom:4 }}>{
          <div style={{ fontSize:13, color:C.sub, lineHeight:1.65 }}>{children}</di
        </div>
      </div>
    </div>
  );
}

```

```

/* =====
  RECEIPT / CAMERA MODAL
===== */

```

```

function ReceiptModal({ onClose, onAddMany }) {
  const videoRef = useRef(null);
  const canvasRef = useRef(null);
  const streamRef = useRef(null);
  const [phase, setPhase] = useState("cam"); // cam | preview | processing
  const [photo, setPhoto] = useState(null);
  const [results, setResults] = useState([]);
  const [selected, setSelected] = useState({});
  const [expiryMap, setExpiryMap] = useState({});
  const [err, setErr] = useState("");
  const [camErr, setCamErr] = useState("");

  useEffect(() => {
    let active = true;
    async function startCam() {
      try {
        const stream = await navigator.mediaDevices.getUserMedia({

```

```

        video: { facingMode: { ideal:"environment" }, width:{ ideal:1920 }, hei
    });
    if (!active) { stream.getTracks().forEach(t => t.stop()); return; }
    streamRef.current = stream;
    if (videoRef.current) { videoRef.current.srcObject = stream; videoRef.cur
    } catch {
        setCamErr("Kamera nicht verfügbar. Bitte Erlaubnis erteilen und nochmal v
    }
    }
    startCam();
    return () => { active = false; streamRef.current?.getTracks().forEach(t => t.
}, []);

```

```

function capture() {
    const v = videoRef.current;
    const cv = canvasRef.current;
    if (!v || !cv) return;
    cv.width = v.videoWidth || 1280;
    cv.height = v.videoHeight || 720;
    cv.getContext("2d").drawImage(v, 0, 0);
    const url = cv.toDataURL("image/jpeg", 0.85);
    setPhoto(url);
    streamRef.current?.getTracks().forEach(t => t.stop());
    setPhase("preview");
}

```

```

function retake() {
    setPhoto(null); setPhase("cam"); setErr("");
    navigator.mediaDevices.getUserMedia({ video:{ facingMode:{ ideal:"environment
        .then(stream => {
            streamRef.current = stream;
            if (videoRef.current) { videoRef.current.srcObject = stream; videoRef.cur
        })
        .catch(() => setCamErr("Kamera nicht verfügbar."));
    }
}

```

```

async function analyse() {
    if (!photo) return;
    setPhase("processing"); setErr("");
    const base64 = photo.split(",")[1];
    const prompt = `Du bist ein Lebensmittel-Experte. Analysiere diesen Kassenzet

```

Extrahiere NUR Lebensmittel und Getränke. Ignoriere komplett: Zahnpasta, Shampoo,

Schätze für jeden Artikel eine sinnvolle Kategorie aus: Milchprodukte, Fleisch, G

Antworte NUR mit einem validen JSON-Array ohne Markdown:

```
[{"name":"Produktname","cat":"Kategorie","qty":"1","unit":"Stk"}]
```

Falls keine Lebensmittel erkennbar sind, antworte mit: []`;

```
try {
  const res = await fetch("https://api.anthropic.com/v1/messages", {
    method: "POST",
    headers: { "Content-Type":"application/json" },
    body: JSON.stringify({
      model: MODEL,
      max_tokens: 1500,
      messages: [{
        role: "user",
        content: [
          { type:"image", source:{ type:"base64", media_type:"image/jpeg", da
            { type:"text", text: prompt }
          ]
        }
      }]
    })
  });
  const data = await res.json();
  const raw = data.content?.find(b => b.type === "text")?.text || "[]";
  const parsed = JSON.parse(raw.replace(/```json|```/g,"").trim());
  if (!Array.isArray(parsed) || parsed.length === 0) {
    setErr("Keine Lebensmittel erkannt. Bitte den Kassenzettel deutlicher fot
    setPhase("preview"); return;
  }
  const defaultExpiry = daysFromNow(14);
  setResults(parsed);
  const sel = {}; parsed.forEach((_, i) => { sel[i] = true; });
  setSelected(sel);
  const exp = {}; parsed.forEach((_, i) => { exp[i] = defaultExpiry; });
  setExpiryMap(exp);
  setPhase("results");
} catch {
  setErr("Fehler bei der Analyse. Bitte nochmal versuchen.");
  setPhase("preview");
}
}

function confirm() {
  const toAdd = results
    .filter((_, i) => selected[i])
    .map((item, i) => ({ ...item, expiry: expiryMap[i] || daysFromNow(14) }));
  if (toAdd.length > 0) onAddMany(toAdd);
  onClose();
}
```

```
const selectedCount = results.filter((_, i) => selected[i]).length;

return (
  <div style={{ position:"fixed", inset:0, zIndex:100, background:"#000",
    display:"flex", flexDirection:"column" }} className="fade-in">
    <div style={{ height:"env(safe-area-inset-top,44px)", background:"#000" }}

      {/* header bar */}

      <div style={{ display:"flex", justifyContent:"space-between",
        alignItems:"center", padding:"12px 16px", background:"#000" }}>
        <div style={{ fontSize:16, fontWeight:800, color:"#fff" }}>
          {{ cam:"Kassenzettel fotografieren", preview:"Foto prüfen",
            processing:"Analysiere...", results:"Erkannte Lebensmittel" }}[phase]
        </div>
        <button onClick={onClose} className="tap"
          style={{ background:"rgba(255,255,255,0.12)", border:"none",
            borderRadius:9, padding:"8px", cursor:"pointer", display:"flex" }}>
          <Ic n="x" s={18} col="#fff" />
        </button>
      </div>

      {/* camera phase */}
      {phase === "cam" && (
        <div style={{ flex:1, position:"relative", display:"flex", flexDirection:
          {camErr ? (
            <div style={{ flex:1, display:"flex", flexDirection:"column",
              alignItems:"center", justifyContent:"center", padding:32, gap:16 }}
              <Ic n="camera" s={48} col="rgba(255,255,255,0.3)" />
              <div style={{ color:"rgba(255,255,255,0.7)", fontSize:14,
                textAlign:"center", lineHeight:1.6 }}>{camErr}</div>
            </div>
          ) : (
            <>
              <video ref={videoRef} playsInline muted
                style={{ flex:1, width:"100%", objectFit:"cover", background:"#11
              <div style={{ position:"absolute", inset:0, display:"flex",
                flexDirection:"column", alignItems:"center", justifyContent:"cent
                pointerEvents:"none" }}>
                <div style={{ border:"2px solid rgba(255,255,255,0.6)", borderRad
                  width:"85%", height:"55%", boxShadow:"0 0 0 2000px rgba(0,0,0,0
                <div style={{ marginTop:16, background:"rgba(0,0,0,0.65)",
                  borderRadius:20, padding:"8px 16px" }}>
                  <span style={{ color:"rgba(255,255,255,0.9)", fontSize:12, font
                    Kassenzettel im Rahmen ausrichten
                  </span>
                </div>
```



```

        </div>
    </>
  )}
  <canvas ref={canvasRef} style={{ display:"none" }} />
  <div style={{ padding:"24px 24px calc(24px + env(safe-area-inset-bottom
    background:"#000", display:"flex", justifyContent:"center" }}>
    <button onClick={capture} disabled={!camErr} className="tap"
      style={{ width:72, height:72, borderRadius:"50%",
        background: camErr ? "#333" : "#fff",
        border:"4px solid rgba(255,255,255,0.3)", cursor: camErr ? "not-a
        display:"flex", alignItems:"center", justifyContent:"center",
        boxShadow:"0 0 0 2px rgba(255,255,255,0.5)" }}>
      <div style={{ width:54, height:54, borderRadius:"50%",
        background: camErr ? "#555" : "#eee", border:"3px solid #ccc" }}
    </button>
  </div>
</div>
)}

/* preview / processing phase */
{(phase === "preview" || phase === "processing") && photo && (
  <div style={{ flex:1, display:"flex", flexDirection:"column", background:
    <div style={{ flex:1, overflow:"hidden", display:"flex",
      alignItems:"center", justifyContent:"center" }}>
      <img src={photo} alt="Kassenzettel"
        style={{ maxWidth:"100%", maxHeight:"100%", objectFit:"contain" }}
    </div>
    {err && (
      <div style={{ background:"#7f1d1d", margin:"0 16px 8px",
        borderRadius:12, padding:"12px 14px" }}>
        <div style={{ color:"#fecaca", fontSize:13 }}>{err}</div>
      </div>
    )}
    <div style={{ padding:"16px 20px calc(20px + env(safe-area-inset-bottom
      background:"#000", display:"flex", gap:10 }}>
      <button onClick={retake} disabled={phase === "processing"} className=
        style={{ flex:1, background:"rgba(255,255,255,0.1)", border:"none",
          borderRadius:13, padding:"14px", color:"#fff", fontSize:14,
          fontWeight:700, cursor: phase==="processing" ? "not-allowed" : "p
          fontFamily:"inherit" }}>
        Nochmal
      </button>
      <button onClick={analyse} disabled={phase === "processing"} className
        style={{ flex:2, background: phase==="processing"
          ? "#333" : `linear-gradient(135deg,${C.green},#1a5c32)` ,
          border:"none", borderRadius:13, padding:"14px", color:"#fff",
          fontSize:14, fontWeight:700,

```

```
cursor: phase=="processing" ? "not-allowed" : "pointer",
    display:"flex", alignItems:"center", justifyContent:"center",
    gap:8, fontFamily:"inherit" }}>
{phase === "processing"
  ? <><Spinner white /><span>KI analysiert...</span></>
  : <><Ic n="sparkle" s={17} col="#fff" /><span>Lebensmittel erkenn<br/>
</button>
</div>
</div>
)}

{/* results phase */}
{phase === "results" && (
  <div style={{ flex:1, display:"flex", flexDirection:"column", background:C.card, padding:10px}}>
    <div style={{ padding:"12px 20px 10px", background:C.card, borderBottom:`1px solid ${C.border}` }}>
      <div style={{ fontSize:13, color:C.sub }}>
        <strong style={{ color:C.text }}>{selectedCount}</strong> von {results.length} Ergebnissen  

        Ablaufdatum anpassen falls nötig
      </div>
    </div>
    <div style={{ flex:1, overflowY:"auto", padding:"12px 20px"}}>
      {results.map((item, i) => (
        <div key={i}
          onClick={() => setSelected(s => ({ ...s, [i]: !s[i] })))}
          className="tap"
          style={{ background:C.card, padding:10px, margin:5px, borderRadius:14, display:"flex", alignItems:"center", justify-content:"space-between", cursor:"pointer", transition:"border-color .15s" }}>
            {/* checkbox */}
            <div style={{ width:22, height:22, borderRadius:6, background:selected[i] ? C.green : C.card2, border:`1.5px solid ${selected[i] ? C.green : C.border}`, display:"flex", align-items:"center", justify-content:"center", flexShrink:0, transition:"all .15s" }}>
              {selected[i] && <Ic n="check" s={13} col="#fff" sw={2.5} />}
            </div>
            <div style={{ flex:1, minWidth:0 }}>
              <div style={{ fontWeight:700, fontSize:14 }}>{item.name}</div>
              <div style={{ fontSize:12, color:C.sub, marginTop:1 }}>
                {item.cat} · {item.qty} {item.unit}
              </div>
            </div>
          </div>
        )}
      </div>
      <div style={{ flex:1, paddingTop:10 }}>
        <p>Ablaufdatum anpassen falls nötig</p>
        <input type="text"/>
      </div>
    </div>
```

```

        type="date"
        value={expiryMap[i] || daysFromNow(14)}
        onChange={e => {
            e.stopPropagation();
            setExpiryMap(m => ({ ...m, [i]: e.target.value }));
        }}
        onClick={e => e.stopPropagation()}
        style={{ fontSize:11, color:C.sub, background:"transparent",
            border:"none", outline:"none", cursor:"pointer",
            fontFamily:"inherit", flexShrink:0 }}
    />
</div>
    )})
</div>
<div style={{ padding:"12px 20px calc(16px + env(safe-area-inset-bottom
    background:C.card, borderTop:`1px solid ${C.border}` }}>
    <button onClick={confirm} className="tap"
        style={{ width:"100%",
            background: selectedCount === 0
                ? C.card2 : `linear-gradient(135deg,${C.green},#1a5c32)` ,
            border:"none", borderRadius:14, padding:"15px",
            fontSize:15, fontWeight:800,
            color: selectedCount === 0 ? C.sub : "#fff",
            cursor: selectedCount === 0 ? "not-allowed" : "pointer",
            boxShadow: selectedCount === 0 ? "none" : `0 4px 16px ${C.green}3
            fontFamily:"inherit" }}>
        {selectedCount === 0
            ? "Nichts ausgewählt"
            : `${selectedCount} Produkt${selectedCount > 1 ? "e" : ""} hinzuf
    </button>
</div>
</div>
    )}
</div>
);
}

/* =====
ITEM MODAL
===== */

const CATS = ["Milchprodukte","Fleisch","Gemüse","Obst","Pasta & Reis","Konserve
const UNITS = ["Stk","g","kg","ml","L","Dosen","Kopf","Bund","Päckli"];

function ItemModal({ onClose, onSave, item }) {
    const [f, setF] = useState(
        item || { name:"", cat:"Sonstiges", qty:"1", unit:"Stk", expiry:daysFromNow(1
    );

```

```

const [err, setErr] = useState("");

function save() {
  if (!f.name.trim()) { setErr("Bitte einen Namen eingeben."); return; }
  onSave(f);
  onClose();
}

return (
  <Sheet onClose={onClose} title={item ? "Produkt bearbeiten" : "Produkt hinzuf
    {err && <ErrMsg msg={err} />}
    <FieldLabel>Name</FieldLabel>
    <input value={f.name}
      onChange={e => { setF(p => ({ ...p, name:e.target.value })); setErr("");
      placeholder="z.B. Vollmilch"
      style={{ ...inputStyle(false), marginBottom:14 }} />

    <FieldLabel>Ablaufdatum</FieldLabel>
    <input type="date" value={f.expiry}
      onChange={e => setF(p => ({ ...p, expiry:e.target.value })))}
      style={{ ...inputStyle(false), marginBottom:14 }} />

    <div style={{ display:"grid", gridTemplateColumns:"1fr 1fr", gap:10, margin
      <div>
        <FieldLabel>Menge</FieldLabel>
        <input type="number" value={f.qty}
          onChange={e => setF(p => ({ ...p, qty:e.target.value })))}
          style={inputStyle(false)} />
      </div>
      <div>
        <FieldLabel>Einheit</FieldLabel>
        <select value={f.unit} onChange={e => setF(p => ({ ...p, unit:e.target.
          style={inputStyle(false)}>
          {UNITS.map(u => <option key={u}>{u}</option>)}
        </select>
      </div>
    </div>

    <FieldLabel>Kategorie</FieldLabel>
    <select value={f.cat} onChange={e => setF(p => ({ ...p, cat:e.target.value
      style={{ ...inputStyle(false), marginBottom:20 }}>
      {CATS.map(c => <option key={c}>{c}</option>)}
    </select>

    <PrimaryBtn onClick={save} text={item ? "Speichern" : "Hinzufügen"} col={C.
  </Sheet>
);

```

```
}
```

```
/* =====  
    ADD RECIPE FORM  
===== */
```

```
function RecipeForm({ onClose, onSave }) {  
  const [f, setF] = useState({  
    name:"", time:30, diff:"Einfach", diet:"vegetarisch", ings:"", stepsText:"",  
  });  
  const [err, setErr] = useState("");  
  
  function save() {  
    if (!f.name.trim()) { setErr("Bitte einen Rezeptnamen eingeben."); return; }  
    if (!f.ings.trim()) { setErr("Bitte mindestens eine Zutat eingeben."); return  
    onSave({  
      ...f,  
      ing:   f.ings.split("\n").filter(Boolean),  
      steps: f.stepsText.split("\n").filter(Boolean),  
    });  
    onClose();  
  }  
}
```

```
return (  
  <Sheet onClose={onClose} title="Rezept erstellen">  
    {err && <ErrMsg msg={err} />}  
  
    <FieldLabel>Name</FieldLabel>  
    <input value={f.name}  
      onChange={e => { setF(p => ({ ...p, name:e.target.value })); setErr("");  
      placeholder="z.B. Gemüse-Curry"  
      style={{ ...inputStyle(false), marginBottom:14 }} />  
  
    <div style={{ display:"grid", gridTemplateColumns:"1fr 1fr 1fr", gap:8, mar  
      <div>  
        <FieldLabel>Zeit (Min)</FieldLabel>  
        <input type="number" value={f.time}  
          onChange={e => setF(p => ({ ...p, time:+e.target.value })))  
          style={inputStyle(false)} />  
      </div>  
      <div>  
        <FieldLabel>Schwierigkeit</FieldLabel>  
        <select value={f.diff} onChange={e => setF(p => ({ ...p, diff:e.target.  
          style={inputStyle(true)}>  
          [{"Einfach","Mittel","Schwer"}.map(d => <option key={d}>{d}</option>  
        </select>  
      </div>  
    </div>
```

```

        <FieldLabel>Ernährung</FieldLabel>
        <select value={f.diet} onChange={e => setF(p => ({ ...p, diet:e.target.
          style={inputStyle(true)})>
          [{"vegetarisch","vegan","fleisch"].map(d => <option key={d}>{d}</opti
        </select>
      </div>
    </div>

    <FieldLabel>Zutaten (eine pro Zeile)</FieldLabel>
    <textarea value={f.ings}
      onChange={e => { setF(p => ({ ...p, ings:e.target.value })); setErr("");
      rows={3} placeholder={"400g Spaghetti\n2 Eier\n1 Zwiebel"}
      style={{ ...inputStyle(false), resize:"none", marginBottom:14 }} />

    <FieldLabel>Zubereitungsschritte (einer pro Zeile)</FieldLabel>
    <textarea value={f.stepsText}
      onChange={e => setF(p => ({ ...p, stepsText:e.target.value })))}
      rows={3} placeholder={"Wasser kochen\nPasta al dente kochen\nSauce zubere
      style={{ ...inputStyle(false), resize:"none", marginBottom:20 }} />

    <PrimaryBtn onClick={save} text="Rezept speichern" col={C.green} />
  </Sheet>
);
}

/* =====
  AI RECIPE MODAL
  ===== */

function AiModal({ onClose, items, onSave }) {
  const [time, setTime] = useState(30);
  const [diet, setDiet] = useState("egal");
  const [diff, setDiff] = useState("Einfach");
  const [loading, setLoading] = useState(false);
  const [err, setErr] = useState("");

  async function generate() {
    setLoading(true); setErr("");
    const list = items.map(i => i.name).join(", ");
    const prompt = `Du bist ein Schweizer Küchenchef. Erstelle ein reales, lecker
Maximalzeit: ${time} Minuten. Ernährung: ${diet === "egal" ? "beliebig" : diet}.
Basics (Salz, Pfeffer, Wasser) kannst du voraussetzen.
Antworte NUR mit einem validen JSON-Objekt, ohne Markdown:
{"name":"...", "time":20, "diff":"Einfach", "diet":"vegetarisch", "ing":["Menge Zutat

    try {
      const res = await fetch("https://api.anthropic.com/v1/messages", {
        method: "POST",

```

```

    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      model: MODEL, max_tokens: 1200,
      messages: [{ role: "user", content: prompt }]
    }),
  });
const data = await res.json();
const text = data.content?.find(b => b.type === "text")?.text || "";
const recipe = JSON.parse(text.replace(/```json|```/g, "").trim());
onSave(recipe);
} catch {
  setErr("Fehler beim Generieren. Bitte nochmal versuchen.");
  setLoading(false);
}
}

return (
  <Sheet onClose={onClose} title="KI-Rezept generieren">
    <div style={{ background:C.greenL, border:`1px solid ${C.greenM}`,
      borderRadius:12, padding:"13px 14px", marginBottom:18,
      fontSize:13, color:C.sub, lineHeight:1.6 }}>
      KI analysiert deine{" "}
      <strong style={{ color:C.green }}>{items.length} Produkte</strong>{" "}
      und kocht dir ein passendes Rezept zusammen.
    </div>

    <div style={{ marginBottom:16 }}>
      <div style={{ fontSize:12, fontWeight:700, color:C.sub, marginBottom:8 }}
        Maximalzeit: {time} Minuten
      </div>
      <input type="range" min={10} max={120} step={5} value={time}
        onChange={e => setTime(+e.target.value)}
        style={{ width:"100%", accentColor:C.green }} />
      <div style={{ display:"flex", justifyContent:"space-between",
        fontSize:11, color:C.sub, marginTop:3 }}>
        <span>10 Min</span><span>120 Min</span>
      </div>
    </div>

    <div style={{ display:"grid", gridTemplateColumns:"1fr 1fr", gap:10, margin
      <div>
        <FieldLabel>Ernährung</FieldLabel>
        <select value={diet} onChange={e => setDiet(e.target.value)} style={inp
          [{"egal", "vegetarisch", "vegan", "fleisch"].map(d => <option key={d}>{d
        </select>
      </div>
    </div>

```

```

        <FieldLabel>Schwierigkeit</FieldLabel>
        <select value={diff} onChange={e => setDiff(e.target.value)} style={inp
          [{"Einfach", "Mittel", "Schwer"}].map(d => <option key={d}>{d}</option>)
        </select>
      </div>
    </div>

    {err && <ErrMsg msg={err} />}
    <PrimaryBtn onClick={generate} text={<><Ic n="sparkle" s={18} col="#fff" />
      col={C.green} loading={loading} />
  </Sheet>
);
}

/* =====
  RECIPE VIEW
  ===== */
function RecipeView({ onClose, recipe }) {
  return (
    <Sheet onClose={onClose} title={recipe.name}>
      <div style={{ display:"flex", gap:7, flexWrap:"wrap", marginBottom:20 }}>
        <Chip icon="clock" text={` ${recipe.time} Min`} col={C.orange} />
        <Chip text={recipe.diff} col={C.sub} />
        {recipe.diet && <Chip icon="leaf" text={recipe.diet} col={C.green} />}
        {recipe.gen && <Chip icon="sparkle" text="KI generiert" col={C.purple} />}
      </div>

      <div style={{ fontSize:11, fontWeight:700, color:C.sub,
        textTransform:"uppercase", letterSpacing:"0.1em", marginBottom:12 }}>Zuta
      {recipe.ing?.map((ing, i) => (
        <div key={i} style={{ display:"flex", alignItems:"center", gap:10,
          padding:"9px 0", borderBottom:`1px solid ${C.border}` }}>
          <div style={{ width:6, height:6, borderRadius:"50%",
            background:C.green, flexShrink:0 }} />
          <span style={{ fontSize:14 }}>{ing}</span>
        </div>
      )))

      <div style={{ fontSize:11, fontWeight:700, color:C.sub,
        textTransform:"uppercase", letterSpacing:"0.1em",
        marginTop:22, marginBottom:14 }}>Zubereitung</div>
      {recipe.steps?.map((step, i) => (
        <div key={i} style={{ display:"flex", gap:13, marginBottom:16 }}>
          <div style={{ width:28, height:28, borderRadius:"50%",
            background:C.orangeL, border:`1.5px solid ${C.orange}40`,
            display:"flex", alignItems:"center", justifyContent:"center",
            flexShrink:0, fontSize:12, fontWeight:800, color:C.orange }}>{i+1}</div>

```



```

        <div style={{ fontSize:14, lineHeight:1.65, paddingTop:4 }}>{step}</div>
    </div>
    )))
  </Sheet>
);
}

/* =====
    ROOT APP
    ===== */

export default function App() {
  const [tab,      setTab]      = useState("home");
  const [items,    setItems]    = useState(SEED_ITEMS);
  const [recipes, setRecipes]   = useState(SEED_RECIPES);
  const [nid,      setNid]      = useState(500);
  const [modal,    setModal]    = useState(null);

  const soon = items.filter(i => daysLeft(i.expiry) <= 3);

  function newId() { const id = nid; setNid(id + 1); return id; }

  function addItem(item) {
    setItems(p => [...p, { ...item, id:newId(), added:todayStr() }]);
  }
  function addItems(arr) {
    let base = nid; setNid(base + arr.length);
    setItems(p => [...p, ...arr.map((it,i) => ({ ...it, id:base+i, added:todayStr
  }
  function updateItem(u) { setItems(p => p.map(i => i.id === u.id ? u : i)); }
  function removeItem(id){ setItems(p => p.filter(i => i.id !== id)); }
  function addRecipe(r) {
    const id = newId();
    const full = { ...r, id };
    setRecipes(p => [...p, full]);
    return full;
  }

  return (
    <div style={{ fontFamily:"'Outfit',sans-serif", background:C.bg,
      minHeight:"100dvh", maxWidth:430, margin:"0 auto",
      position:"relative", overflowX:"hidden", color:C.text }}>
      <style>{`
        @import url('https://fonts.googleapis.com/css2?family=Outfit:wght@400;500
        *{box-sizing:border-box;margin:0;padding:0;-webkit-tap-highlight-color:tr
        ::-webkit-scrollbar{display:none;}
        input,textarea,select{font-family:'Outfit',sans-serif;}
        @keyframes fu{from{opacity:0;transform:translateY(12px)}to{opacity:1;tran

```

```

@keyframes spin{to{transform:rotate(360deg)}}
@keyframes slideUp{from{transform:translateY(100%);opacity:0}to{transform
@keyframes fadeIn{from{opacity:0}to{opacity:1}}
.fu{animation:fu .26s cubic-bezier(.22,1,.36,1) both;}
.slide-up{animation:slideUp .3s cubic-bezier(.22,1,.36,1) both;}
.fade-in{animation:fadeIn .2s ease both;}
.tap:active{transform:scale(.96);transition:transform .1s;}
`}</style>

<div style={{ height:"env(safe-area-inset-top,44px)", background:C.card }}

<header style={{ padding:"12px 20px 10px", background:C.card,
borderBottom:`1px solid ${C.border}`, position:"sticky", top:0, zIndex:40
boxShadow:"0 1px 3px rgba(0,0,0,0.06)" }}>
<div style={{ display:"flex", justifyContent:"space-between", alignItems:
<div>
<div style={{ fontSize:24, fontWeight:900, letterSpacing:"-0.8px", li
<span style={{ color:C.text }}>fresh</span>
<span style={{ color:C.green }}>ly</span>
</div>
<div style={{ fontSize:11, color:C.sub, fontWeight:500, marginTop:1 }
</div>
{soon.length > 0 && (
<button onClick={() => setTab("pantry")} className="tap"
style={{ background:C.redL, border:`1px solid ${C.red}28`,
borderRadius:10, padding:"7px 12px", display:"flex",
alignItems:"center", gap:5, cursor:"pointer", fontFamily:"inherit
<Ic n="warning" s={14} col={C.red} />
<span style={{ fontSize:12, color:C.red, fontWeight:700 }}>{soon.le
</button>
)}
</div>
</header>

<main style={{ paddingBottom:88 }}>
{tab === "home" && <HomeTab items={items} recipes={recipes} soon=
{tab === "pantry" && <PantryTab items={items} removeItem={removeItem}
{tab === "recipes" && <RecipesTab recipes={recipes} items={items} setMo
{tab === "settings" && <SettingsTab />}
</main>

<BottomNav tab={tab} setTab={setTab} />

{modal?.type === "receipt" && <ReceiptModal onClose={() => setModal(null
{modal?.type === "addItem" && <ItemModal onClose={() => setModal(null
{modal?.type === "addRecipe" && <RecipeForm onClose={() => setModal(null
{modal?.type === "ai" && <AiModal onClose={() => setModal(null

```

```
        {modal?.type === "view"      && <RecipeView      onClose={() => setModal(null  
    </div>  
    );  
}
```